

1. a) Em tipagem estática os tipos das variáveis são conhecidos e verificados em tempo de compilação (antes da execução). O código precisa declarar ou o compilador/analizador infere o tipo e verifica consistência (por exemplo tentar atribuir `string` a `int` gera erro). A verificação é feita por um compilador ou por uma etapa de análise estática que valida atribuições, chamadas de função, etc.

b) **Performance:** o compilador pode gerar código mais otimizado pois conhece os tipos (ex.: operações aritméticas diretas), menos verificações de tipo em tempo de execução.

**Segurança (robustez):** erros de tipo detectados antes da execução reduzem bugs, interfaces e contratos mais claros (APIs com tipos ajudam a evitar uso incorreto). Outros benefícios: melhor autocompletar/IDE, documentação explícita, refatoração mais segura.

c) Em tipagem dinâmica (ex.: JavaScript, Python), variáveis não têm tipo fixo; o tipo é associado ao valor em tempo de execução. As operações verificam tipos enquanto o programa roda — p. ex. concatenar string com número pode converter automaticamente.

Apresenta como desafios de performance: necessidade de checagens em tempo de execução (overhead), otimizações JIT (just-in-time) tentam mitigar, mas a variação de tipos dificulta geração de código tão eficiente quanto em linguagens estáticas, representações internas genéricas aumentam custo de memória/CPU.

d) **Tipagem forte:** conversões implícitas são restritas; operações entre tipos incompatíveis geralmente lançam erro ou não são permitidas sem conversão explícita. (Ex.: Python é relativamente forte.)

**Tipagem fraca:** permite coerções automáticas entre tipos (às vezes surpreendentes). (Ex.: JavaScript faz muitas conversões automáticas: `1 + "2" -> "12"`.)

e) **Híbridas:** oferecem tipagem estática opcional ou sistemas de tipos que não exigem sempre anotações explícitas (ex.: TypeScript sobre JS, Kotlin, Swift). Assim você tem checagem estática quando desejar, mas pode usar dinamismo quando necessário.

**Inferência de tipos:** compilador deduz tipos a partir do contexto sem anotações explícitas (ex.: `let x = 5`  $\rightarrow$  `x: int`). Isso combina segurança estática com sintaxe concisa.

f) **Dinâmica e fraca:** variáveis não obrigam declaração de tipo e os tipos podem mudar (`let x = 5; x = "texto";`), possui tipos primitivos: `number`, `string`, `boolean`, `null`,

`undefined`, `symbol`, `bigint` e objetos (`object`), coerção de tipos automática em várias operações (`==` faz coerção; `===` compara sem coerção), para tipagem estática opcional usa-se **TypeScript**.